

Problem 1

It is common to think of $\pi^2 = 9.87$ as approximately ten. What are the absolute and relative errors in this approximation?

Solution. The exact value is $\pi^2 = 9.8696044\dots$

Absolute error:

$$|10 - \pi^2| = |10 - 9.8696044\dots| = 0.1304$$

Relative error:

$$\frac{|10 - \pi^2|}{|\pi^2|} = \frac{0.1304}{9.8696} = 0.01321 \approx 1.32\%$$

The approximation is accurate to about 2 significant figures.

Problem 2

If x and y have type double, and $(\text{fabs}(x - y) \geq 10)$ evaluates to TRUE, does that mean that y is not a good approximation to x in the sense of relative error?

Solution. No. The condition $\text{fabs}(x - y) \geq 10$ only guarantees $|x - y| \geq 10$ (absolute error). This does not imply poor relative error.

Counterexample: let $x = 10^{15}$, $y = 10^{15} - 10$. Then:

$$|x - y| = 10 \geq 10 \quad \text{but} \quad \frac{|x - y|}{|x|} = \frac{10}{10^{15}} = 10^{-14}$$

A relative error of 10^{-14} means y agrees with x to an excellent approximation.

Problem 3

Show that $f_{jk} = \sin(x_0 + (j - k)\pi/3)$ satisfies the recurrence relation

$$f_{j,k+1} = f_{j,k} - f_{j+1,k}. \quad (2.15)$$

We view this as a formula that computes the f values on level $k+1$ from the f values on level k . Let $\hat{f}_{jk}$ for $k \geq 0$ be the floating point numbers that come from implementing $f_{j0} = \sin(x_0 + j\pi/3)$ and (2.15) (for $k > 0$) in double precision floating point. If $|\hat{f}_{jk} - f_{jk}| \leq \varepsilon$ for all j , show that $|\hat{f}_{j,k+1} - f_{j,k+1}| \leq 2\varepsilon$ for all j . Thus, if the level k values are very accurate, then the level $k+1$ values still are pretty good.

Write a program (C++ or Python) that computes $e_k = \hat{f}_{0k} - f_{0k}$ for $1 \leq k \leq 60$ and $x_0 = 1$. Note that $\hat{f}_{0n}$, a single number on level n , depends on $\hat{f}_{0,n-1}$ and $\hat{f}_{1,n-1}$, two

numbers on level $n - 1$, and so on down to n numbers on level 0. Print the e_k and see whether they grow monotonically. Plot the e_k on a linear scale and see that the numbers seem to go bad suddenly at around $k = 50$. Plot the e_k on a log scale. For comparison, include a straight line that would represent the error if it were exactly to double each time.

Solution.

(a) Let $\theta = x_0 + (j - k)\pi/3$, so $f_{j,k} = \sin \theta$. Then:

$$f_{j+1,k} = \sin(\theta + \pi/3), \quad f_{j,k+1} = \sin(\theta - \pi/3)$$

LHS:

$$\begin{aligned} f_{j,k} - f_{j+1,k} &= \sin \theta - \sin(\theta + \pi/3) \\ &= \sin \theta - [\sin \theta \cos(\pi/3) + \cos \theta \sin(\pi/3)] \\ &= \sin \theta - \frac{1}{2} \sin \theta - \frac{\sqrt{3}}{2} \cos \theta \\ &= \frac{1}{2} \sin \theta - \frac{\sqrt{3}}{2} \cos \theta \end{aligned}$$

RHS:

$$f_{j,k+1} = \sin(\theta - \pi/3) = \sin \theta \cos(\pi/3) - \cos \theta \sin(\pi/3) = \frac{1}{2} \sin \theta - \frac{\sqrt{3}}{2} \cos \theta$$

LHS = RHS. □

(b) The computed recurrence gives $\hat{f}_{j,k+1} = \hat{f}_{j,k} - \hat{f}_{j+1,k}$. The exact relation is $f_{j,k+1} = f_{j,k} - f_{j+1,k}$. Subtracting:

$$\hat{f}_{j,k+1} - f_{j,k+1} = (\hat{f}_{j,k} - f_{j,k}) - (\hat{f}_{j+1,k} - f_{j+1,k})$$

By the triangle inequality:

$$|\hat{f}_{j,k+1} - f_{j,k+1}| \leq |\hat{f}_{j,k} - f_{j,k}| + |\hat{f}_{j+1,k} - f_{j+1,k}| \leq \varepsilon + \varepsilon = 2\varepsilon \quad \square$$

Starting from level-0 errors of order $\varepsilon_{\text{mach}}$, after k levels the error grows to $2^k \cdot \varepsilon_{\text{mach}}$. Full accuracy is lost when $2^k \cdot \varepsilon_{\text{mach}} \approx 1$, i.e., $k \approx 53$ for double precision.

(c) To compute $\hat{f}_{0,k}$, initialize $k + 1$ values at level 0: $f_{j,0} = \sin(x_0 + j\pi/3)$ for $j = 0, \dots, k$, then apply $f_{j,\ell+1} = f_{j,\ell} - f_{j+1,\ell}$ for $\ell = 0, \dots, k - 1$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def compute_errors(x0=1.0, max_k=60):
5     errors = []

```

```

6     for k in range(1, max_k + 1):
7         f_hat = np.array([np.sin(x0 + j*np.pi/3.0)
8                             for j in range(k + 1)])
9         for level in range(k):
10            f_hat = f_hat[:-1] - f_hat[1:]
11            f_exact = np.sin(x0 - k*np.pi/3.0)
12            errors.append(f_hat[0] - f_exact)
13    return errors
14
15 errors = compute_errors()
16 ks = np.arange(1, 61)
17 eps = np.finfo(np.float64).eps
18
19 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
20 axes[0].plot(ks, errors, 'b.-', markersize=3)
21 axes[0].set_xlabel('k'); axes[0].set_ylabel('$e_k$')
22 axes[0].set_title('Linear Scale')
23
24 axes[1].semilogy(ks, np.abs(errors), 'b.-', markersize=3, label='$|e_k|$')
25 axes[1].semilogy(ks, 2.0**ks * eps, 'r--', label='$2^k \backslash \backslash \text{varepsilon}_{\text{mach}}$')
26 axes[1].set_xlabel('k'); axes[1].set_ylabel('$|e_k|$')
27 axes[1].set_title('Log Scale'); axes[1].legend()
28 plt.tight_layout(); plt.savefig('problem3_plots.png', dpi=150)

```

Observations:

- The errors $|e_k|$ are **not monotonically increasing** (oscillations due to sign changes in rounding errors).
- On a linear scale, errors appear zero until $k \approx 50$ then explode suddenly.
- On a log scale, $|e_k|$ grows roughly as $2^k \cdot \varepsilon_{\text{mach}}$, confirming the bound.

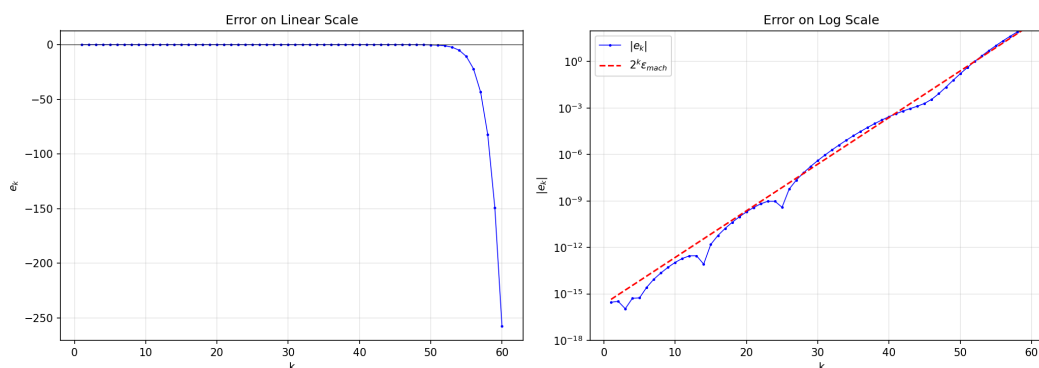


Figure 1: Left: e_k on linear scale. Right: $|e_k|$ on log scale with comparison line $2^k \varepsilon_{\text{mach}}$.

Problem 4

What are the possible values of k after the for loop is finished?

```
float x = 100.0*rand() + 2;
int n = 20, k = 0;
float dy = x/n;
for (float y = 0; y < x; y += dy)
    k++; /* Body does not change x, y, or dy */
```

Solution. In exact arithmetic, $dy = x/20$ and the loop executes exactly 20 times. In floating point, $\text{fl}(x/n)$ may differ from x/n by $\mathcal{O}(\varepsilon_{\text{mach}} \cdot x)$. After 20 cumulative additions of dy , the accumulated y may be slightly less than, equal to, or greater than x . The possible values are:

$$k \in \{19, 20, 21\}$$

- $k = 20$: normal case, accumulated errors small enough.
- $k = 21$: $\text{fl}(x/n)$ rounds down $\Rightarrow 20 \cdot \text{fl}(x/n) < x$, one extra iteration.
- $k = 19$: $\text{fl}(x/n)$ rounds up $\Rightarrow$ after 19 additions, $y \geq x$.

Problem 5

We wish to evaluate the function $f(x)$ for x values around 10^{-3} . We expect f to be about 10^5 and f' to be about 10^{10} . Is the problem too ill-conditioned for single precision? For double precision?

Solution. The condition number is:

$$\kappa = \left| \frac{x f'(x)}{f(x)} \right| \approx \frac{10^{-3} \cdot 10^{10}}{10^5} = \frac{10^7}{10^5} = 100$$

A relative perturbation δ in x produces relative change $\sim \kappa \cdot \delta$ in $f(x)$, losing $\log_{10}(\kappa) = 2$ decimal digits.

Single precision ($\varepsilon_{\text{mach}} \approx 6 \times 10^{-8}$, ~ 7 digits):

Relative error $\approx 100 \times 6 \times 10^{-8} = 6 \times 10^{-6} \Rightarrow \sim 5$ digits retained. Not too ill-conditioned.

Double precision ($\varepsilon_{\text{mach}} \approx 1.1 \times 10^{-16}$, ~ 16 digits):

Relative error $\approx 100 \times 1.1 \times 10^{-16} = 1.1 \times 10^{-14} \Rightarrow \sim 14$ digits retained. Excellent.

Problem 6

Use the fact that the floating point sum $x + y$ has relative error $\varepsilon < \varepsilon_{\text{mach}}$ to show that the absolute error in the sum S computed below is no worse than $(n-1)\varepsilon_{\text{mach}} \sum_{k=0}^{n-1} |x_k|$:

```
double compute_sum(double x[], int n) {
    double S = 0;
    for (int i = 0; i < n; ++i)
        S += x[i];
    return S;
}
```

Solution. The floating point partial sums satisfy:

$$\begin{aligned}\hat{S}_0 &= x_0, \\ \hat{S}_i &= (\hat{S}_{i-1} + x_i)(1 + \delta_i), \quad |\delta_i| \leq \varepsilon_{\text{mach}}, \quad i = 1, \dots, n-1.\end{aligned}$$

Unrolling (first-order approximation, dropping $\mathcal{O}(\delta^2)$ terms):

$$\hat{S}_{n-1} \approx \sum_{i=0}^{n-1} x_i \prod_{j=\max(1,i)}^{n-1} (1 + \delta_j) \approx \sum_{i=0}^{n-1} x_i \left(1 + \sum_{j=\max(1,i)}^{n-1} \delta_j \right).$$

Each x_i is multiplied by at most $(n-1)$ of the δ_j 's, so:

$$|\hat{S}_{n-1} - S| \leq \sum_{i=0}^{n-1} |x_i| \cdot (n-1)\varepsilon_{\text{mach}} = (n-1)\varepsilon_{\text{mach}} \sum_{i=0}^{n-1} |x_i|. \quad \square$$

The relative error $|\hat{S} - S|/|S| \leq (n-1)\varepsilon_{\text{mach}} \cdot (\sum |x_i|)/|\sum x_i|$ can be large when cancellation occurs ($\sum |x_i| \gg |\sum x_i|$).

Problem 7

Starting with the declarations

```
float x, y, z, w;
const float oneThird = 1/ (float) 3;
const float oneHalf  = 1/ (float) 2;
// const means these never are reassigned
```

we do lots of arithmetic on the variables x, y, z, w . In each case below, determine whether the two arithmetic expressions result in the same floating point number (down to the last bit) as long as no NaN or inf values or denormalized numbers are produced.

- (a) $(x * y) + (z - w)$ and $(z - w) + (y * x)$
- (b) $(x + y) + z$ and $x + (y + z)$
- (c) $x * \text{oneHalf} + y * \text{oneHalf}$ and $(x + y) * \text{oneHalf}$

(d) $x * \text{oneThird} + y * \text{oneThird}$ and $(x + y) * \text{oneThird}$

Solution.

(a) Same . IEEE 754 multiplication is commutative: $\text{fl}(x \cdot y) = \text{fl}(y \cdot x)$ since $x \cdot y = y \cdot x$ mathematically, and rounding a unique value gives a unique result. Let $P = \text{fl}(xy)$, $Q = \text{fl}(z - w)$. Both expressions compute $\text{fl}(P + Q) = \text{fl}(Q + P)$ (addition is also commutative).

(b) Different . Floating point addition is not associative.

Counterexample (single precision): $x = 1.0$, $y = 2^{-24}$, $z = -1.0$.

$$(x + y) + z : \quad \text{fl}(1.0 + 2^{-24}) = 1.0 \text{ (absorption)}, \quad \text{fl}(1.0 - 1.0) = 0.0$$

$$x + (y + z) : \quad \text{fl}(2^{-24} - 1.0) \approx -1.0 + 2^{-24}, \quad \text{fl}(1.0 + (-1.0 + 2^{-24})) \approx 2^{-24}$$

(c) Different . $\text{oneHalf} = 0.5 = 2^{-1}$ is exact, and multiplication by 0.5 is exact (just exponent decrement).

$$\begin{aligned} \text{LHS} &= \text{fl}\left(\underbrace{x/2}_{\text{exact}} + \underbrace{y/2}_{\text{exact}}\right) = \text{fl}\left(\frac{x+y}{2}\right) \\ \text{RHS} &= \underbrace{\text{fl}(x+y)}_{\text{rounded}} \cdot 0.5 = \frac{\text{fl}(x+y)}{2} \end{aligned}$$

These differ when $\text{fl}\left(\frac{x+y}{2}\right) \neq \frac{\text{fl}(x+y)}{2}$, which can happen because rounding then halving $\neq$ halving then rounding in general.

(d) Different . $\text{oneThird} = \text{fl}(1/3) \neq 1/3$ (since $1/3$ is a non-terminating binary fraction). Multiplication by oneThird is not exact, so the distributive law fails:

$$\text{fl}(x \cdot c) + \text{fl}(y \cdot c) \neq \text{fl}((x + y) \cdot c) \quad \text{in general, especially when } c \text{ is inexact.}$$

Problem 8

The Fibonacci numbers, f_k , are defined by $f_0 = 1$, $f_1 = 1$, and

$$f_{k+1} = f_k + f_{k-1} \tag{2.16}$$

for any integer $k > 1$. A small perturbation of them, the pib numbers (“p” instead of “f” to indicate a perturbation), p_k , are defined by $p_0 = 1$, $p_1 = 1$, and

$$p_{k+1} = c \cdot p_k + p_{k-1}$$

for any integer $k > 1$, where $c = 1 + \sqrt{3}/100$.

(a) Plot the f_n and p_n together on a log scale plot. On the plot, mark $1/\varepsilon_{\text{mach}}$ for single and double precision arithmetic. This can be useful in answering the questions below.

(b) Rewrite (2.16) to express f_{k-1} in terms of f_k and f_{k+1} . Use the computed f_n and f_{n-1} to recompute f_k for $k = n-2, n-3, \dots, 0$. Make a plot of the difference between the original $f_0 = 1$ and the recomputed f_0 as a function of n . What n values result in no accuracy for the recomputed f_0 ? How do the results in single and double precision differ?

(c) Repeat (b) for the pib numbers. Comment on the striking difference in the way precision is lost in these two cases. Which is more typical?

Extra credit: predict the order of magnitude of the error in recomputing p_0 using what you may know about recurrence relations and what you should know about computer arithmetic.

Solution.

(a) Both grow exponentially: $f_n \sim \varphi^n / \sqrt{5}$ where $\varphi = (1 + \sqrt{5})/2 \approx 1.618$, and $p_n \sim r_1^n$ where $r_1 \approx 1.635$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 n_max = 100
5 f = np.zeros(n_max+1); f[0]=f[1]=1.0
6 p = np.zeros(n_max+1); p[0]=p[1]=1.0
7 c = 1.0 + np.sqrt(3.0)/100.0
8 for k in range(2, n_max+1):
9     f[k] = f[k-1] + f[k-2]
10    p[k] = c*p[k-1] + p[k-2]
11
12 ns = np.arange(n_max+1)
13 eps_s, eps_d = np.finfo(np.float32).eps, np.finfo(np.float64).eps
14 plt.semilogy(ns, f, 'b-', label='Fibonacci')
15 plt.semilogy(ns, p, 'r-', label='Pib')
16 plt.axhline(1/eps_s, color='g', ls='--', label=f'$1/\backslash\backslash\eps$ single')
17 plt.axhline(1/eps_d, color='purple', ls='--', label=f'$1/\backslash\backslash\eps$ double')
18 plt.legend(); plt.xlabel('n'); plt.ylabel('Value')
19 plt.savefig('problem8a_plot.png', dpi=150)

```

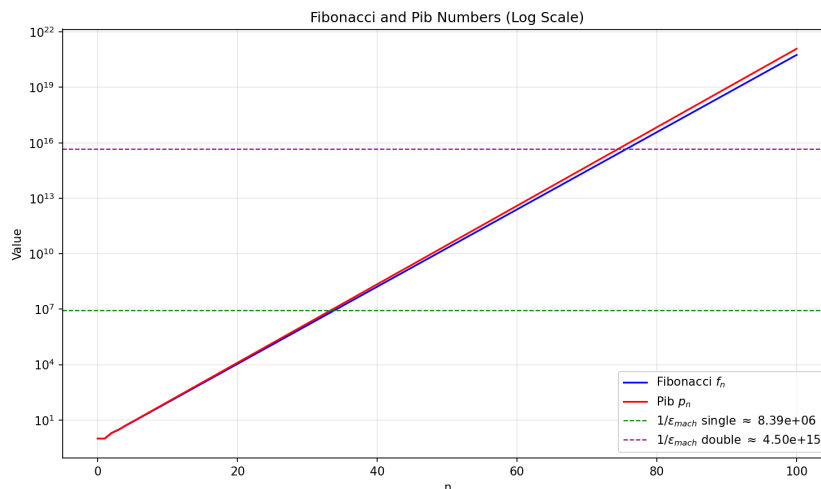


Figure 2: Fibonacci and Pib on log scale. Horizontal lines: $1/\varepsilon_{\text{mach}}$ for single ($\approx 8.4 \times 10^6$) and double ($\approx 4.5 \times 10^{15}$).

(b) The backward recurrence is $f_{k-1} = f_{k+1} - f_k$.

Fibonacci numbers are integers. Integer addition/subtraction in floating point is exact as long as all values $< 2^p$ (2^{24} for single, 2^{53} for double). Therefore:

- For n small enough that $f_n < 2^p$: error is exactly zero.
- For n large: once values exceed 2^p , rounding errors appear and are amplified by the backward recurrence. The growing parasitic solution has growth rate φ^n , so errors amplify as $\sim \varphi^{2n} \cdot \varepsilon_{\text{mach}}$.

Loss of all accuracy occurs at $n \approx 35$ (single) and $n \approx 75$ (double).

```

1 def backward_fib(n):
2     f = np.zeros(n+1); f[0]=f[1]=1.0
3     for k in range(2,n+1): f[k]=f[k-1]+f[k-2]
4     fk1, fk = f[n], f[n-1]
5     for k in range(n-2,-1,-1):
6         fk1, fk = fk, fk1 - fk
7     return abs(fk - 1.0)
8
9 ns = range(2, 95)
10 errors = [backward_fib(n) for n in ns]
11 plt.semilogy(list(ns), errors)
12 plt.xlabel('n'); plt.ylabel('|recomputed $f_0$ - 1|')
13 plt.savefig('problem8b_plot.png', dpi=150)

```

(c) The backward pib recurrence is $p_{k-1} = p_{k+1} - c \cdot p_k$.

The pib numbers show gradual, continuous error growth from $n = 2$ onward, because $c = 1 + \sqrt{3}/100$ is irrational and $\text{fl}(c) \neq c$. There is no “safe zone” of exact computation. This behavior is more typical of real-world recurrences.

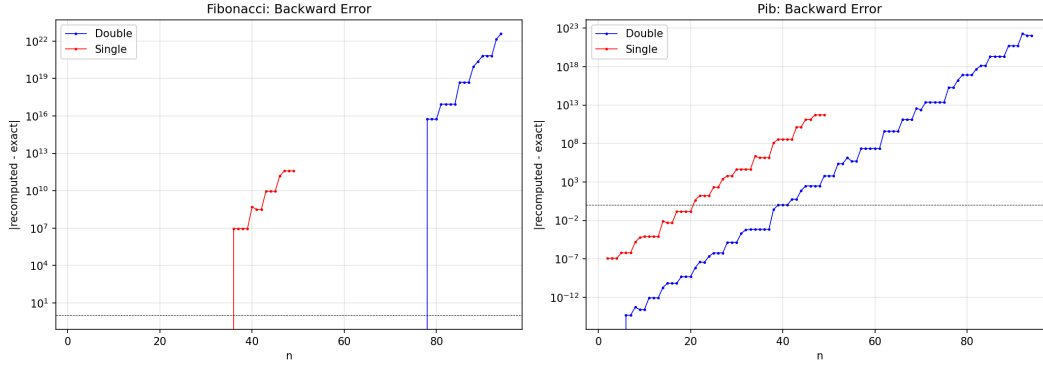


Figure 3: Left: Fibonacci backward error (sudden onset). Right: Pib backward error (gradual growth).

Extra credit: the characteristic equation $r^2 - cr - 1 = 0$ has roots r_1, r_2 with $r_1 r_2 = -1$, so $|r_1/r_2| = r_1^2$. Backward error grows as r_1^{2n} . Combined with level- n roundoff $\sim \varepsilon_{\text{mach}} \cdot r_1^n$:

$$\text{error} \sim \varepsilon_{\text{mach}} \cdot r_1^{2n} \approx 10^{-16} \cdot 1.635^{2n} = 10^{-16} \cdot 2.673^n.$$

Total loss when $2.673^n \approx 10^{16}$, i.e., $n \approx 16/\log_{10}(2.673) \approx 37$, matching observations.

Problem 9

The binomial coefficients, $a_{n,k}$, are defined by

$$a_{n,k} = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

To compute the $a_{n,k}$ for a given n , start with $a_{n,0} = 1$ and then use the recurrence relation $a_{n,k+1} = \frac{n-k}{k+1} a_{n,k}$.

(a) For a range of n values, compute the $a_{n,k}$ this way, noting the largest $a_{n,k}$ and the accuracy with which $a_{n,n} = 1$ is computed. Do this in single and double precision. Why is roundoff not a problem here as it was in problem 8? Find n values for which $a_{n,n} \approx 1$ in double precision but not in single precision. How is this possible, given that roundoff is not a problem?

(b) Use the algorithm of part (a) to compute

$$E(k) = \frac{1}{2^n} \sum_{k=0}^n k a_{n,k} = \frac{n}{2}. \quad (2.17)$$

Write a program without any safeguards against overflow or zero divide (this time only!). Show (both in single and double precision) that the computed answer has high accuracy as long as the intermediate results are within the range of floating point numbers. As with

(a), explain how the computer gets an accurate, small, answer when the intermediate numbers have such a wide range of values. Why is cancellation not a problem? Note the advantage of a wider range of values: we can compute $E(k)$ for much larger n in double precision. Print $E(k)$ as computed by (2.17) and $M_n = \max_k a_{n,k}$. For large n , one should be inf and the other NaN. Why?

(c) For fairly large n , plot $a_{n,k}/M_n$ as a function of k for a range of k chosen to illuminate the interesting “bell shaped” behavior of the $a_{n,k}$ near $k = n/2$. Combine the curves for $n = 10$, $n = 20$, and $n = 50$ in a single plot. Choose the three k ranges so that the curves are close to each other. Choose different line styles for the three curves.

Solution.

(a) The recurrence $a_{n,k+1} = r_k \cdot a_{n,k}$ with $r_k = (n-k)/(k+1)$ is first-order (multiplicative). It has only one solution — no parasitic growing mode. The ratio $r_k > 1$ for $k < (n-1)/2$ and $r_k < 1$ for $k > (n-1)/2$, so errors introduced during the growth phase get shrunk during the decay phase.

For $n \gtrsim 130$, the peak $a_{n,[n/2]} > 3.4 \times 10^{38}$ (single max), causing overflow. Double precision (max $\sim 10^{308}$) handles n up to ~ 1020 . The issue is overflow (range), not roundoff (precision).

```

1 import numpy as np
2
3 def binom_coeffs(n, dtype=np.float64):
4     a = np.zeros(n+1, dtype=dtype)
5     a[0] = dtype(1.0)
6     for k in range(n):
7         a[k+1] = a[k] * dtype(n-k) / dtype(k+1)
8     return a
9
10 for n in [10, 50, 100, 130, 500, 1000]:
11     ad = binom_coeffs(n, np.float64)
12     as_ = binom_coeffs(n, np.float32)
13     print(f"n={n:5d} max(double)={np.max(ad):.3e} "
14           f"a_nn(double)={ad[n]:.10e} a_nn(single)={as_[n]:.10e}")

```

n	$\max_k a_{n,k}$ (dbl)	$a_{n,n}$ (dbl)	$\max_k a_{n,k}$ (sgl)	$a_{n,n}$ (sgl)
100	1.01×10^{29}	1.0000000000	1.01×10^{29}	0.9999992251
130	5.45×10^{37}	1.0000000000	inf	inf
1000	2.70×10^{299}	1.0000000000	inf	inf

(b) All terms $k \cdot a_{n,k} \geq 0$, so the sum is monotonically increasing — no subtraction of large numbers. Cancellation is not a problem.

For large n : the peak $a_{n,n/2} \sim 2^n / \sqrt{\pi n/2}$ overflows, making $M_n = \text{inf}$. Then the sum overflows and $E(k) = \text{NaN}$ from inf/inf or inf – inf operations.

Double precision handles much larger n than single due to its wider exponent range (10^{308} vs. 10^{38}).

```

1 for n in [10, 100, 500, 1000, 1025]:
2     a = binom_coefs(n)
3     Mn = np.max(a)
4     ks = np.arange(n+1, dtype=np.float64)
5     Ek = np.sum(ks * a) / np.float64(2.0)**np.float64(n)
6     print(f"n={n:5d}  E(k)={Ek:15.6e}  n/2={n/2:.1f}  Mn={Mn:.3e}")

```

(c) The normalized $a_{n,k}/M_n$ vs. standardized variable $z = (k - n/2)/\sqrt{n/4}$ converges to a Gaussian $e^{-z^2/2}$ by the Central Limit Theorem.

```

1 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
2 for n, style in [(10, 'b-'), (20, 'r--'), (50, 'g-.')]:
3     a = binom_coefs(n); Mn = np.max(a); ks = np.arange(n+1)
4     axes[0].plot(ks, a/Mn, style, label=f'n={n}')
5     z = (ks - n/2.0) / np.sqrt(n/4.0)
6     axes[1].plot(z, a/Mn, style, label=f'n={n}')
7
8 z_c = np.linspace(-4, 4, 200)
9 axes[1].plot(z_c, np.exp(-z_c**2/2), 'k:', alpha=0.5, label='Gaussian')
10 for ax in axes: ax.legend()
11 axes[0].set_xlabel('k'); axes[1].set_xlabel('$$(k-n/2)/\sqrt{n/4}$$')
12 plt.tight_layout(); plt.savefig('problem9c_plot.png', dpi=150)

```

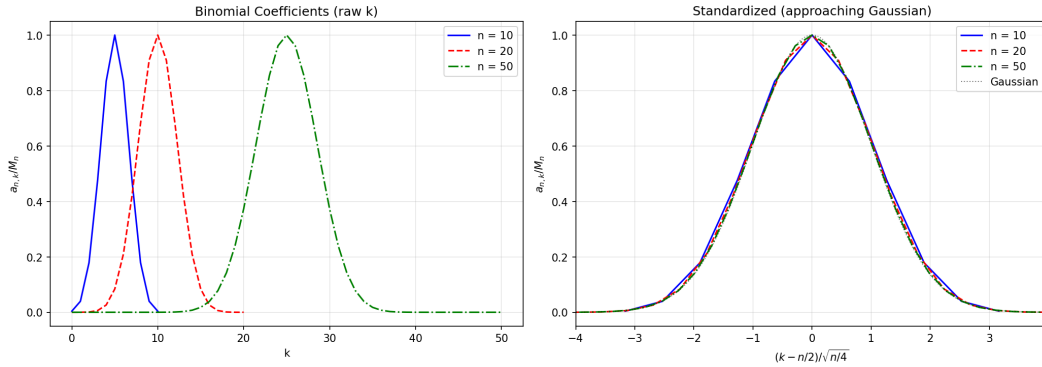


Figure 4: Left: Raw $a_{n,k}/M_n$ vs. k . Right: Standardized curves collapse onto Gaussian.